

The BM25 Formula, Worked Out by Hand

The ranking function that has powered search engines for thirty years and is still the keyword half of every hybrid RAG system — derived term by term and computed in full on two documents, including why a longer document with the same words scores lower.

What this document does. It assembles BM25 from three parts you can read independently — the IDF weight, the saturating term-frequency factor, and the length-normalisation denominator — then scores a query against two documents by hand, showing every multiplication. The two documents contain the *same* query-term counts but differ in length, so the length term does visible work. Numbers from Appendix A.

Prerequisite: TF-IDF / IDF (#14). The k_1 and b knobs get their own treatment in #16.

1 The formula

For a query Q and document d of length $|d|$ in a corpus with average length avgdl , BM25 sums one contribution per query term:

$$\text{BM25}(Q, d) = \sum_{t \in Q} \text{idf}(t) \cdot \frac{\text{tf}(t, d) (k_1 + 1)}{\text{tf}(t, d) + k_1 (1 - b + b \frac{|d|}{\text{avgdl}})}$$

Three ideas, multiplied:

- $\text{idf}(t)$ — rare terms count more (PDF #14). The *between-document* signal.
- the numerator $\text{tf}(k_1+1)$ with tf in the denominator — a *saturating* reward for repetition. The *within-document* signal.
- $1 - b + b |d|/\text{avgdl}$ — penalises length, so a long document cannot win just by containing more words.

Intuition

BM25 is TF-IDF with two fixes learned from decades of search. Fix 1: the tenth occurrence of a word should not count as much as the first — relevance *saturates*. So tf sits in both numerator and denominator, bending the curve toward a ceiling of $\text{idf} \cdot (k_1+1)$. Fix 2: a 10,000-word page will naturally contain query terms more often than a tweet; divide that advantage out by comparing each document's length to the corpus average. What remains is an honest “aboutness” score.

2 The pieces, isolated

Saturation (the tf factor). Hold $\text{idf} = 1$, $b = 0$. As tf grows the factor $\frac{\text{tf}(k_1+1)}{\text{tf}+k_1}$ rises toward $k_1 + 1$ but never passes it — diminishing returns baked in. (Full table in PDF #16.)

Length normalisation (the b factor). The bracket $1 - b + b |d|/\text{avgdl}$ equals 1 when $|d| = \text{avgdl}$ (no effect at average length), exceeds 1 for long documents (shrinking their scores), and is below 1 for short ones. b scales how hard this bites; $b = 0$ disables it, $b = 1$ applies it fully.

3 Worked by hand

Query **"network security"**. Corpus: $N = 10$ documents, $\text{avgdl} = 12$ tokens. Parameters $k_1 = 1.5$, $b = 0.75$ (the standard defaults). Document frequencies $\text{df}(\text{network}) = 3$, $\text{df}(\text{security}) = 2$, giving (BM25 IDF from #14):

$$\text{idf}(\text{network}) = 1.1451, \quad \text{idf}(\text{security}) = 1.4816.$$

Document D_1 : length 10, $\text{tf}(\text{network}) = 3$, $\text{tf}(\text{security}) = 1$. Length factor uses $|d|/\text{avgdl} = 10/12$:

$$K = k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right) = 1.5 \left(0.25 + 0.75 \cdot \frac{10}{12}\right) = 1.5(0.875) = 1.3125.$$

Term **network** ($\text{tf} = 3$):

$$1.1451 \cdot \frac{3(2.5)}{3 + 1.3125} = 1.1451 \cdot \frac{7.5}{4.3125} = 1.9915.$$

Term **security** ($\text{tf} = 1$):

$$1.4816 \cdot \frac{1(2.5)}{1 + 1.3125} = 1.4816 \cdot \frac{2.5}{2.3125} = 1.6017.$$

$$\text{BM25}(Q, D_1) = 1.9915 + 1.6017 = \boxed{3.5933}.$$

Document D_2 : identical term counts, length 40. Same tf values, but $|d|/\text{avgdl} = 40/12$:

$$K = 1.5 \left(0.25 + 0.75 \cdot \frac{40}{12}\right) = 1.5(2.75) = 4.125.$$

$$\text{network: } 1.1451 \cdot \frac{7.5}{3 + 4.125} = 1.2054, \quad \text{security: } 1.4816 \cdot \frac{2.5}{1 + 4.125} = 0.7227,$$

$$\text{BM25}(Q, D_2) = 1.2054 + 0.7227 = \boxed{1.9281}.$$

Mini-example • same words, longer doc, lower score

D_1 and D_2 contain **network** three times and **security** once — *exactly the same evidence*. Yet D_1 (length 10, near average) scores 3.59 while D_2 (length 40, far above average) scores 1.93 — nearly half. The length term decided it: in a 40-word document, three mentions of **network** is dilute; in a 10-word document it is dense and on-topic. This length-awareness is BM25's single biggest improvement over raw TF-IDF.

4 Why BM25 still matters

BM25 needs no training, no embeddings, and no GPU; it is exact, explainable, and unbeatable at one thing dense vectors are weak at — *precise lexical matches* like error codes, identifiers, names, and rare jargon (CVE-2024-31082 must match *exactly*, and an embedding may “round it off”). That is why production RAG rarely chooses BM25 *or* dense retrieval and instead fuses both (hybrid search, Track 3.2): BM25 catches the exact tokens, dense catches the paraphrases.

Equation cheat-sheet

$$\text{BM25} = \sum_{t \in Q} \text{idf}(t) \cdot \frac{\text{tf}(k_1 + 1)}{\text{tf} + k_1(1 - b + b|d|/\text{avgdl})}$$

saturates at $\text{idf}(k_1+1)$; $|d| = \text{avgdl} \Rightarrow \text{length factor} = 1$; defaults $k_1=1.5$, $b=0.75$

Appendix A · Reference implementation

```
import numpy as np
N, avgdl, k1, b = 10, 12.0, 1.5, 0.75
df = {"network": 3, "security": 2}
idf = lambda t: np.log((N - df[t] + 0.5) / (df[t] + 0.5) + 1)

def bm25(tf, dl):
    s = 0.0
    for t, f in tf.items():
        K = k1 * (1 - b + b * dl / avgdl)
        s += idf(t) * (f * (k1 + 1)) / (f + K)
    return s

bm25({"network":3, "security":1}, 10)    # 3.5933 (D1)
bm25({"network":3, "security":1}, 40)    # 1.9281 (D2, same words, 4x longer)
```

Internal learning material. Numbers reproducible from the appendix code. v1.0